Understanding Java generics, type erasure, and wildcards

https://ms.sg.dev/deepsearch/a7aa620e-6f27-402f-abb6-327b4115db89

---

1)What is Generics in Java? How it works? explain in detail 2)Whats type erasure, upper bound, lower bound? 3)When to use which of them? 4)Why wild card needed when either of above are available

---

Java Generics — Complete Guide

---

1. What are Generics and How Do They Work?

Generics allow you to write **type-parameterized** classes, interfaces, and methods — so you write the logic once and it works for any type, with **compile-time type safety**.

Without Generics (pre Java 5)

```
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0); // manual cast — ClassCastException risk at runtime
```

With Generics

```
List<String> list = new ArrayList<>();
list.add("hello");
String s = list.get(0); // no cast needed, compiler enforces type
```

Generic Class

```
public class Box<T> {           // T is a type parameter (placeholder)
    private T value;

    public void set(T value) { this.value = value; }
    public T get() { return value; }
```

```
}

Box<Integer> intBox = new Box<>();
intBox.set(42);
Integer val = intBox.get();     // no cast, fully type-safe
```

Generic Method

```
public <T> T pickFirst(T[] items) {
    return items[0];
}

String first = pickFirst(new String[]{"a", "b"}); // T inferred as String
```

Multiple Type Parameters

```
public class Pair<K, V> {
    private K key;
    private V value;
    // ...
}

Pair<String, Integer> p = new Pair<>("age", 30);
```

How the Compiler Enforces It

The compiler inserts type checks and casts automatically, and **rejects** incompatible types at compile time — not at runtime.

```
List<String> names = new ArrayList<>();
names.add(123); // COMPILE ERROR — not a String
```

---

2. Type Erasure, Upper Bound, Lower Bound

Type Erasure

At **runtime**, Java removes all generic type information. This is called **type erasure** — it exists for **backward compatibility** with pre-generics bytecode (Java 4 and older).

What happens at compile time:

```
List<String>  --> becomes --> List (raw type) in bytecode
List<Integer> --> becomes --> List (raw type) in bytecode
```

The compiler:

1. Replaces `T` with `Object` (or the bound if bounded)
2. Inserts casts where needed
3. Generates bridge methods for polymorphism

Proof — at runtime, generics are invisible:

```
List<String> strings = new ArrayList<>();
List<Integer> ints = new ArrayList<>();

System.out.println(strings.getClass() == ints.getClass()); // true — both are ArrayList
```

Consequences of Type Erasure

Restriction	Reason
Cannot do `new T()`	T is unknown at runtime
Cannot do `T[] arr = new T[10]`	Array creation needs exact type
Cannot do `instanceof List<String>`	Type param erased at runtime
Cannot overload on `List<String>` vs `List<Integer>`	Same erasure = same signature

```
// NOT allowed — both erase to the same method signature
void process(List<String> list) {}
void process(List<Integer> list) {} // COMPILE ERROR
```

Upper Bound — `<? extends T>`

Restricts the type to **T or any subtype of T**. Think: "I accept T or anything that IS-A T."

```
public double sumList(List<? extends Number> list) {
    double sum = 0;
    for (Number n : list) {
        sum += n.doubleValue(); // safe — guaranteed to be a Number
    }
    return sum;
}
```

```
sumList(new ArrayList<Integer>());  // OK — Integer extends Number
sumList(new ArrayList<Double>());   // OK — Double extends Number
sumList(new ArrayList<String>());   // COMPILE ERROR
```

Rule: You can READ from an upper-bounded wildcard, but NOT write (except null).

```
List<? extends Number> list = new ArrayList<Integer>();
Number n = list.get(0);   // OK — reading is safe
list.add(42);             // COMPILE ERROR — compiler doesn't know if it's List<Integer>, List<Double>, etc.
```

Lower Bound — `<? super T>`

Restricts the type to **T or any supertype of T**. Think: "I accept T or anything that T IS-A."

```
public void addNumbers(List<? super Integer> list) {
    list.add(1);
    list.add(2);
    list.add(3);
}

addNumbers(new ArrayList<Integer>());  // OK
addNumbers(new ArrayList<Number>());   // OK — Integer is-a Number
addNumbers(new ArrayList<Object>());   // OK — Integer is-a Object
addNumbers(new ArrayList<Double>());   // COMPILE ERROR — Integer is NOT a Double
```

Rule: You can WRITE to a lower-bounded wildcard, but reading gives you only `Object`.

```
List<? super Integer> list = new ArrayList<Number>();
list.add(42);              // OK — Integer is safe to insert
Object o = list.get(0);   // only Object — compiler can't be more specific
```

---

3. When to Use Which?

The PECS Principle (Producer Extends, Consumer Super)

> **If a collection PRODUCES (you read from it) → use `extends` If a collection CONSUMES (you write to it) → use `super`**

Scenario	Use
Reading elements from a collection	`<? extends T>`
Writing elements into a collection	`<? super T>`
Both reading and writing	`<T>` (concrete type parameter)
Completely independent of type	`<?>` (unbounded wildcard)

Concrete Type Parameter `<T>`

Use when you need to **relate types** across parameters or return type:

```
// T ties the input type to the return type
public <T> T identity(T value) { return value; }

// T ensures both lists have the same type
public <T> void copy(List<T> source, List<T> dest) { ... }
```

Upper Bound `<? extends T>`

Use for **read-only / producer** collections:

```
// Reading to compute — classic upper bound use case
public static double sum(List<? extends Number> numbers) {
    return numbers.stream().mapToDouble(Number::doubleValue).sum();
}
```

Lower Bound `<? super T>`

Use for **write-only / consumer** collections:

```
// Writing into a collection — classic lower bound use case
public static void fill(List<? super Integer> list, int count) {
    for (int i = 0; i < count; i++) list.add(i);
}
```

Unbounded Wildcard `<?>`

Use when the type **does not matter at all**:

```
public void printAll(List<?> list) {
    for (Object o : list) System.out.println(o); // only Object methods available
}
```

---

4. Why Are Wildcards Needed When Bounds Exist on Type Parameters?

This is the most subtle part. The key insight is **use-site flexibility vs. declaration-site rigidity**.

The Problem Wildcards Solve

Generics are **invariant** in Java. This surprises many people:

```
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;   // COMPILE ERROR — even though Integer extends Number
```

This is intentional — if it were allowed:

```
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;   // hypothetically allowed
nums.add(3.14);             // adds a Double into a List<Integer> — type corruption!
Integer x = ints.get(0);   // ClassCastException at runtime
```

`<T extends Number>` vs `<? extends Number>` — The Difference

`<T extends Number>` on a method declaration means: "caller fixes T to one specific type, and the whole method is consistent with that type."

```
public <T extends Number> void process(List<T> list, T item) {
    list.add(item); // safe — T is fixed for both
}
process(new ArrayList<Integer>(), 42);   // T = Integer
process(new ArrayList<Double>(), 3.14);  // T = Double
```

`<? extends Number>` means: "I don't know or care what the exact type is — I just know it's some subtype of Number."

```
public void process(List<? extends Number> list) {
    // list.add(42); // COMPILE ERROR — unknown exact type
    Number n = list.get(0); // OK — safe to read
}
// Accepts List<Integer>, List<Double>, List<Float> — all at the call site
```

When You CANNOT Use `<T>` and MUST Use `<?>`

```
// Goal: accept a List<List<Integer>>, List<List<String>>, etc.

// With T — too rigid, T binds to one type only
public <T> void printNested(List<List<T>> lists) { }
printNested(new ArrayList<List<Integer>>());  // OK
// but you can't pass mixed or unknown nested lists

// With wildcard — truly flexible
public void printNested(List<? extends List<?>> lists) { }
// accepts any list of lists, regardless of inner type
```

Side-by-Side Summary

Feature	`<T>`	`<? extends T>`	`<? super T>`	`<?>`
Type is fixed	Yes, by caller	No	No	No
Can read as	`T`	`T`	`Object` only	`Object` only
Can write	Yes	No (except null)	Yes	No (except null)
Relates parameters	Yes	No	No	No
Use at declaration	Yes	Yes	Yes	Yes
Use at call site flexibility	Low	High	High	Highest
PECS role	Both	Producer	Consumer	Don't care

The Real Reason Wildcards Exist

```
// Without wildcards, you'd need method overloads for every subtype:
void sum(List<Integer> list) { ... }
void sum(List<Double> list) { ... }
void sum(List<Float> list) { ... }

// With wildcards — one method handles all:
void sum(List<? extends Number> list) { ... }
```

Wildcards enable **covariance and contravariance at the use site**, which `<T>` alone cannot express, because `<T>` always requires an exact type match.

Quick Reference

```
<T>              — exact known type, needed for type relationships
<T extends Foo>  — upper-bounded type param, can read as Foo, can also relate params
<? extends Foo>  — upper-bounded wildcard, read-only (producer), max flexibility
<? super Foo>    — lower-bounded wildcard, write-only (consumer)
<?>              — unbounded, completely type-agnostic operations only
```

Explain about type erasure? when it happens, how

Type Erasure in Java — Deep Dive

What is Type Erasure?

Type erasure is the process by which the **Java compiler removes all generic type information** from the compiled bytecode (`.class` files). After compilation, generic types become raw types — as if generics were never written.

It was introduced this way deliberately in Java 5 to maintain **backward binary compatibility** with Java 4 and earlier libraries.

When Does It Happen?

At compile time — during the compilation phase, not at runtime.

```
Your .java source          javac compiler              .class bytecode
─────────────────    ──────────────────────────    ──────────────────────
List<String>      →  type-checks generics        →  List  (raw type)
Box<Integer>      →  inserts casts where needed  →  Box   (raw type)
```

```
<T extends Foo>   →  replaces T with Foo         →  Foo
<T>               →  replaces T with Object      →  Object
```

The JVM **never sees** generic types. It only sees raw types + casts.

How It Happens — Step by Step

The compiler performs **3 transformations** during erasure:

Step 1 — Replace Type Parameters with Their Bound (or Object)

Rule:

- Unbounded `<T>` → replaced with `Object`
- Bounded `<T extends Foo>` → replaced with `Foo` (the leftmost bound)

```
// SOURCE — what you write
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}
```

```
// BYTECODE EQUIVALENT — what the compiler produces
public class Box {
    private Object value;                    // T → Object
    public void set(Object value) { this.value = value; }
    public Object get() { return value; }    // T → Object
}
```

With a bound:

```
// SOURCE
public class NumberBox<T extends Number> {
    private T value;
    public T get() { return value; }
}
```

```
// BYTECODE EQUIVALENT
public class NumberBox {
```

```
    private Number value;         // T → Number (the bound, not Object)
    public Number get() { return value; }
}
```

With multiple bounds:

```
// SOURCE
public class Foo<T extends Comparable<T> & Serializable> {
    T data;
}
```

```
// BYTECODE EQUIVALENT — leftmost bound wins
public class Foo {
    Comparable data;              // T → Comparable (leftmost bound)
}
```

Step 2 — Insert Casts Automatically

Because `get()` now returns `Object` (or the erased bound), the compiler **inserts casts at every call site** automatically to preserve type safety.

```
// SOURCE
Box<String> box = new Box<>();
box.set("hello");
String s = box.get();           // looks like no cast needed
```

```
// BYTECODE EQUIVALENT — compiler inserted a cast
Box box = new Box();
box.set("hello");
String s = (String) box.get();  // cast inserted by compiler
```

You never write the cast — the compiler writes it for you, and it is **guaranteed to be safe** because it already type-checked your source.

Step 3 — Generate Bridge Methods (for Polymorphism)

This is the least-known part of erasure. When a generic class is subclassed with a concrete type, erasure can **break polymorphism** — so the compiler generates invisible "bridge methods" to fix it.

```
// SOURCE
public class StringBox extends Box<String> {
    @Override
    public void set(String value) {   // overrides Box<String>.set(String)
        System.out.println("Setting: " + value);
        super.set(value);
    }
}
```

After erasure, `Box` has `set(Object)`. But `StringBox` has `set(String)`. These are **different signatures** — polymorphism is broken. So the compiler silently generates:

```
// BYTECODE EQUIVALENT — bridge method added by compiler
public class StringBox extends Box {

    // Your actual method (after erasure of String → String, no change here)
    public void set(String value) {
        System.out.println("Setting: " + value);
        super.set(value);
    }

    // Bridge method — generated by compiler, invisible to you
    public void set(Object value) {       // matches erased Box.set(Object)
        this.set((String) value);         // delegates to your real method + cast
    }
}
```

You can actually see bridge methods using `javap -verbose`:

```
javap -verbose StringBox.class
# flags: ACC_PUBLIC, ACC_BRIDGE, ACC_SYNTHETIC  ← bridge method marker
```

Proving Type Erasure at Runtime

```
List<String> strings = new ArrayList<>();
List<Integer> ints    = new ArrayList<>();

// Both are just ArrayList at runtime — type param is gone
System.out.println(strings.getClass() == ints.getClass());  // true
System.out.println(strings.getClass().getName());            // java.util.ArrayList

// Cannot distinguish at runtime
```

```
System.out.println(strings instanceof List<String>);  // COMPILE ERROR — illegal
System.out.println(strings instanceof List<?>);       // OK — wildcard allowed
```

What You CANNOT Do Because of Type Erasure

Each restriction maps directly to erasure:

```
public class Demo<T> {

    // 1. Cannot instantiate T — type is unknown at runtime
    T obj = new T();                    // COMPILE ERROR

    // 2. Cannot create generic arrays
    T[] arr = new T[10];               // COMPILE ERROR
    // Workaround:
    T[] arr2 = (T[]) new Object[10];   // allowed but unchecked warning

    // 3. Cannot use instanceof with parameterized type
    if (obj instanceof List<String>) { } // COMPILE ERROR — erased

    // 4. Cannot catch or throw generic exceptions
    // class MyException<T> extends Exception { } // not allowed
    // try { } catch (T e) { }                    // not allowed

    // 5. Cannot overload on erased signatures
    void process(List<String> list) { }
    void process(List<Integer> list) { } // COMPILE ERROR — same erasure
}
```

Heap Pollution — When Erasure Goes Wrong

Heap pollution occurs when a variable of a parameterized type refers to an object that is **not of that type** — a direct consequence of erasure.

```
List<String> strings = new ArrayList<>();
List rawList = strings;          // raw type assignment — compiles with warning
rawList.add(42);                 // adds Integer into a List<String> — no error yet

// Erasure means the list has no runtime guard:
String s = strings.get(0);      // ClassCastException HERE — at get(), not at add()
```

The cast inserted by the compiler at `strings.get(0)` is what blows up — not the `add()`. This is confusing because the error appears far from its cause.

`@SuppressWarnings("unchecked")` — What It Means

When you do something the compiler cannot verify due to erasure, it issues an **unchecked warning**. Suppressing it is a promise from you that you've manually verified safety:

```
@SuppressWarnings("unchecked")
public <T> T deserialize(Object raw) {
    return (T) raw;   // unchecked cast — T is erased, compiler can't verify
}
```

Never suppress this blindly — it silences a real warning about potential `ClassCastException`.

Reifiable vs. Non-Reifiable Types

Type	Reifiable?	Meaning
`int`, `String`, `Object`	Yes	Full type info available at runtime
`List<?>`, `Map<?,?>`	Yes	Wildcards are reifiable
`List<String>`	**No**	Erased to `List` at runtime
`T`, `E`, `K`	**No**	Type params are erased

Reifiable = type information fully available at runtime. **Non-reifiable** = type information partially or fully erased.

Arrays require reifiable types — this is why `new List<String>[10]` is illegal:

```
List<String>[] arr = new List<String>[10];  // COMPILE ERROR — generic array creation
List<?>[] arr2     = new List<?>[10];       // OK — wildcard is reifiable
```

Summary — The Full Picture

```
SOURCE (.java)                    BYTECODE (.class)
```

```
Box<T>                        →    Box  (T replaced with Object)
Box<T extends Number>         →    Box  (T replaced with Number)
List<String> list             →    List list
String s = list.get(0)        →    String s = (String) list.get(0)  ← cast injected
@Override set(String value)   →    set(String) + bridge set(Object)  ← bridge injected
instanceof List<String>       →    ILLEGAL (non-reifiable)
new T[10]                     →    ILLEGAL (non-reifiable)
```

One-line summary:

> The compiler uses generics for type-checking at compile time, then strips them out entirely — replacing type params with bounds (or Object), injecting casts at call sites, and generating bridge methods for polymorphism — so the JVM runs the same bytecode it always did.